LAPORAN PRATIKUM
STRUKTUR DATA
“QUEUE”

Dosen Pengampu:

DR. Wahyudi. .S.T.M.T

Asisten Lab:

Jovantri Immanuel Gulo

Disusun Oleh:

Marcello Bayu Denar Widjaksono

2511532011

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data, dengan judul: “QUEUE”.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan program komputer, pemahaman mendalam tentang struktur data (data structure) menjadi fondasi utama bagi para programmer karena menentukan efisiensi penyimpanan dan pengolahan informasi. Salah satu pendekatan yang paling umum digunakan adalah Abstract Data Type (ADT), yaitu model logis yang mendefinisikan perilaku data beserta operasi yang dapat dilakukan tanpa bergantung pada implementasi fisik tertentu, sehingga memudahkan pengelolaan dan pemeliharaan kode. Pada bahasa pemrograman Java, konsep ADT Queue direpresentasikan melalui antarmuka Queue dalam paket java.util, yang secara praktis diimplementasikan menggunakan kelas seperti LinkedList atau ArrayDeque. Queue merupakan struktur data linear yang bekerja berdasarkan prinsip FIFO (First In First Out), di mana elemen yang pertama kali dimasukkan ke dalam struktur akan menjadi elemen pertama yang dikeluarkan. Berbeda dengan Array atau List yang memungkinkan akses elemen secara acak melalui indeks, Queue membatasi operasi manipulasi data secara ketat: penambahan elemen hanya terjadi di ujung belakang (rear/tail), sedangkan penghapusan hanya dilakukan di ujung depan (front/head). Pembatasan akses ini justru menjadi keunggulan Queue karena menjamin keterurutan dan mencegah konflik data pada aliran yang berkesinambungan. Sifat terstruktur tersebut membuat Queue sangat efisien dan banyak diterapkan dalam berbagai skenario komputasi nyata, antara lain penjadwalan proses dan thread pada sistem operasi, manajemen antrian pencetakan dokumen (printer spooling), algoritma traversal graf seperti Breadth-First Search (BFS), serta penanganan buffer pada komunikasi jaringan, message queue, dan sistem streaming data.

1.2 Tujuan Pratikum

1. Menjelaskan konsep dan karakteristik Queue sebagai implementasi struktur data ADT dengan prinsip FIFO dalam Java.Mengidentifikasi metode-metode utama Stack (seperti push, pop, peek, isEmpty, dan size) beserta mekanisme kerjanya.
2. Mengidentifikasi metode-metode utama Queue (seperti add/offer, remove/poll, element/peek, isEmpty, dan size) beserta mekanisme kerjanya.

1.3 Manfaat Pratikum

1. Meningkatkan pemahaman konsep struktur data bertipe FIFO melalui penggunaan Queue dalam bahasa pemrograman Java.
2. Mampu menggunakan metode-metode utama Queue untuk menyelesaikan permasalahan komputasi yang relevan.

BAB II
PEMBAHASAN

2.1 Pratikum Striktur Data

```java
package pekan4_2511532011;

import java.util.Queue;

public class QueueArray_2511532011 {
	int front_2011, rear_2011, size_2011;
	int capacity_2011;
	int array_2011[];

	public QueueArray_2511532011(int capacity) {
		this.capacity_2011 = capacity;
		front_2011 = this.size_2011 = 0;
		rear_2011 = capacity - 1;
		array_2011 = new int [this.capacity_2011];
	}

	boolean isFull_2011(QueueArray_2511532011 queue) {
		return (queue.size_2011 == queue.capacity_2011);
	}

	boolean isEmpty_2011 (QueueArray_2511532011 queue) {
		return (queue.size_2011 == 0);
	}

	void enqueue_2011(int item_2011) {
		if (isFull_2011(this))
			return;
		this.rear_2011 = (this.rear_2011 + 1) % this.capacity_2011;
		this.array_2011[this.rear_2011 ] = item_2011;
		this.size_2011 = this.size_2011 + 1;
		System.out.println(item_2011 + " enqueued to queue");
	}

	int dequeue_2011() {
		if(isEmpty_2011(this))
			return Integer.MIN_VALUE;
		int item_2011 = this.array_2011[this.front_2011];
		this.front_2011 = (this.front_2011 + 1) % this.capacity_2011;
		this.size_2011 = this.size_2011 - 1;
		return item_2011;
	}

	int front_2011() {
		if (isEmpty_2011(this))
			return Integer.MIN_VALUE;

		return this.array_2011[this.front_2011];
	}

	int rear_2011() {
		if (isEmpty_2011(this))
			return Integer.MIN_VALUE;
		return this.array_2011[this.rear_2011];
	}

	void display_2011() {
		int i;
		if (front_2011 == rear_2011) {
			System.out.printf("\nantrian kosong\n");
			return;
		}

		for ( i = front_2011; i < rear_2011; i++) {
			System.out.printf(" %d <--", array_2011[i]);
		}

		return;
	}

}
```

Pembahasan Class QueueArray:

1. Deklarasi Package dan Import

   package pekan4_2511532011;: Menetapkan namespace kelas agar terorganisir dalam struktur folder proyek praktikum minggu ke-4.

   import java.util.Queue;: Mengimpor interface Queue dari library standar Java. Meskipun diimpor, kelas ini membuat implementasi manual sendiri tanpa mengimplementasikan interface tersebut secara langsung.

2. Deklarasi Kelas dan Atribut

   public class QueueArray_2511532011: Mendefinisikan kelas publik sebagai cetak biru untuk objek antrian.

   int front_2011, rear_2011, size_2011;: Variabel untuk melacak index depan (head), index belakang (tail), dan jumlah elemen saat ini.

   int capacity_2011;: Variabel untuk menyimpan kapasitas maksimum antrian.

   int array_2011[]: Deklarasi array integer yang akan berfungsi sebagai penyimpanan fisik elemen antrian.

3. Konstruktor (Inisialisasi)

   public QueueArray_2511532011(int capacity): Konstruktor yang menerima parameter ukuran kapasitas antrian.

   this.capacity_2011 = capacity;: Mengassign nilai kapasitas ke variabel instance.

   front_2011 = this.size_2011 = 0;: Menginisialisasi index depan dan ukuran awal menjadi 0 (kosong).

   rear_2011 = capacity - 1;: Menginisialisasi index belakang di posisi terakhir array. Ini adalah teknik khusus agar saat enqueue pertama kali dilakukan, index rear menjadi 0 setelah operasi modulo.

   array_2011 = new int [this.capacity_2011];: Alokasi memori untuk array dengan ukuran sesuai kapasitas.

4. Metode isFull_2011

   boolean isFull_2011(QueueArray_2511532011 queue): Metode untuk memeriksa apakah antrian penuh.

return (queue.size_2011 == queue.capacity_2011);: Mengembalikan true jika jumlah elemen saat ini sama dengan kapasitas maksimum, menandakan antrian penuh.

5. Metode isEmpty_2011

   boolean isEmpty_2011 (QueueArray_2511532011 queue): Metode untuk memeriksa apakah antrian kosong.

   return (queue.size_2011 == 0);: Mengembalikan true jika tidak ada elemen sama sekali dalam antrian.

6. Metode enqueue_2011 (Masuk Antrian)

   void enqueue_2011(int item_2011): Metode untuk menambahkan elemen baru ke belakang antrian.

   if (isFull_2011(this)) return;: Kondisi pencegahan (guard clause). Jika antrian penuh, metode berhenti tanpa melakukan apa-apa.

   this.rear_2011 = (this.rear_2011 + 1) % this.capacity_2011;: Logika Circular. Index rear incremented dan dilakukan operasi modulo kapasitas. Ini memastikan jika index mencapai akhir array, ia akan kembali ke 0 (wrap-around).

   this.array_2011[this.rear_2011] = item_2011;: Menyimpan nilai item ke dalam array pada posisi rear yang baru.

   this.size_2011 = this.size_2011 + 1;: Menambah counter ukuran antrian.

   System.out.println(...): Mencetak konfirmasi bahwa data berhasil masuk.

7. Metode dequeue_2011 (Keluar Antrian)

   int dequeue_2011(): Metode untuk menghapus dan mengembalikan elemen dari depan antrian.

   if(isEmpty_2011(this)) return Integer.MIN_VALUE;: Jika antrian kosong, mengembalikan nilai minimum integer sebagai tanda error/kosong.

   int item_2011 = this.array_2011[this.front_2011];: Menyimpan nilai elemen depan ke variabel sementara sebelum dihapus.

   this.front_2011 = (this.front_2011 + 1) % this.capacity_2011;: Logika Circular. Menggeser index depan ke berikutnya dengan wrap-around menggunakan modulo.

   this.size_2011 = this.size_2011 - 1;: Mengurangi counter ukuran antrian.

return item_2011;: Mengembalikan nilai elemen yang telah dihapus.

8. Metode front_2011 dan rear_2011 (Peek)

   int front_2011(): Mengambil nilai elemen depan tanpa menghapusnya.

   int rear_2011(): Mengambil nilai elemen belakang tanpa menghapusnya.

   Kedua metode memiliki pemeriksaan if (isEmpty_2011(this)) untuk mencegah akses data pada antrian kosong, mengembalikan Integer.MIN_VALUE jika kosong.

   return this.array_2011[...]: Mengakses array langsung berdasarkan index front atau rear.

9. Metode display_2011 (Tampilan)

   void display_2011(): Metode untuk mencetak isi antrian ke konsol.

   if (front_2011 == rear_2011): Kondisi pemeriksaan kosong berdasarkan perbandingan index (catatan: dalam implementasi circular queue dengan variabel size, pemeriksaan size == 0 lebih akurat, namun kode ini menggunakan perbandingan index).

   System.out.printf("\nantrian kosong\n");: Pesan jika kondisi kosong terpenuhi.

   for (i = front_2011; i < rear_2011; i++): Perulangan untuk iterasi dari index depan hingga belakang.

   System.out.printf(" %d <--", array_2011[i]);: Mencetak setiap elemen dengan panah untuk menunjukkan urutan antrian.

```java
package pekan4_2511532011;

public class QueueArrayDriver_2511532011 {
        public static void main ( String [] args) {
                QueueArray_2511532011 queue_2011 = new QueueArray_2511532011(1000);

                queue_2011.enqueue_2011(10);
                queue_2011.enqueue_2011(20);
                queue_2011.enqueue_2011(30);
                queue_2011.enqueue_2011(40);

                System.out.println("item di depan " + queue_2011.front_2011());
                System.out.println("item di paling belakang " + queue_2011.rear_2011());
                System.out.println("tampilkan queue");

                queue_2011.display_2011();

                System.out.println();
                System.out.println(queue_2011.dequeue_2011() + " dihapus dari queue");
                System.out.println("item di depan :" + queue_2011.front_2011());
                System.out.println("item di  belakang :" + queue_2011.rear_2011());

                System.out.println("tampilkan queue setelah dihapus");
                queue_2011.display_2011();
        }
}
```

Pembahasan Class QueueArrayDriver:

1. Deklarasi Package

   package pekan4_2511532011;: Menetapkan namespace kelas agar berada dalam direktori yang sama dengan kelas QueueArray_2511532011 untuk memastikan aksesibilitas antar kelas.

2. Deklarasi Kelas

   public class QueueArrayDriver_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai driver atau pengujian utama untuk kelas antrian yang telah dibuat sebelumnya.

3. Metode Utama (Main)

   public static void main (String [] args): Titik masuk (entry point) program dimana eksekusi kode dimulai.

4. Instansiasi Objek Antrian

   QueueArray_2511532011 queue_2011 = new QueueArray_2511532011(1000);:

   Membuat objek baru dari kelas QueueArray_2511532011.

   Mengirimkan nilai 1000 sebagai argumen konstruktor, yang berarti antrian ini dialokasikan dengan kapasitas maksimum sebanyak 1000 elemen integer.

Variabel queue_2011 adalah referensi untuk mengakses metode-metode antrian.

5. Operasi Enqueue (Masuk Antrian)

   queue_2011.enqueue_2011(10);: Menambahkan nilai 10 ke dalam antrian (posisi paling belakang).

   queue_2011.enqueue_2011(20);: Menambahkan nilai 20 setelah 10.

   queue_2011.enqueue_2011(30);: Menambahkan nilai 30 setelah 20.

   queue_2011.enqueue_2011(40);: Menambahkan nilai 40 setelah 30.

   State Antrian: Saat ini antrian berisi [10, 20, 30, 40] dengan front menunjuk ke 10 dan rear menunjuk ke 40.

6. Operasi Peek (Lihat Depan dan Belakang)

   System.out.println("item di depan " + queue_2011.front_2011());:

   Memanggil metode front_2011() untuk mengambil nilai elemen paling depan tanpa menghapusnya.

   Expected Output: item di depan 10.

   System.out.println("item di paling belakang " + queue_2011.rear_2011());:

   Memanggil metode rear_2011() untuk mengambil nilai elemen paling belakang.

   Expected Output: item di paling belakang 40.

7. Operasi Display (Tampilan Awal)

   System.out.println("tampilkan queue");: Mencetak label informasi.

   queue_2011.display_2011();: Memanggil metode untuk mengiterasi dan mencetak seluruh elemen yang ada dalam antrian saat ini secara berurutan.

8. Operasi Dequeue (Keluar Antrian)

   System.out.println();: Mencetak baris baru untuk kerapian output.

   System.out.println(queue_2011.dequeue_2011() + " dihapus dari queue");:

   Memanggil metode dequeue_2011() yang menghapus elemen paling depan (10) dan mengembalikan nilainya.

   Nilai yang dikembalikan dicetak bersama pesan konfirmasi.

   State Antrian: Antrian kini berisi [20, 30, 40]. Index front bergeser ke elemen berikutnya.

9. Verifikasi Setelah Dequeue

   System.out.println("item di depan :" + queue_2011.front_2011());:

   Memeriksa elemen depan setelah penghapusan. Karena 10 sudah dihapus, elemen depan sekarang adalah 20.

   System.out.println("item di belakang :" + queue_2011.rear_2011());:

   Memeriksa elemen belakang. Posisi belakang tidak berubah, masih 40.

10. Operasi Display (Tampilan Akhir)

    System.out.println("tampilkan queue setelah dihapus");: Mencetak label informasi status terkini.

    queue_2011.display_2011();: Mencetak sisa elemen dalam antrian (20, 30, 40) untuk memverifikasi bahwa elemen 10 telah berhasil dihapus dari struktur data.

```java
package pekan4_2511532011;

import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511532011 {
	public static void main (String [] args) {
		Queue<Integer> q_2011 = new LinkedList<>();

		for ( int i = 0; i < 6; i++)
			q_2011.add(i);

		System.out.println("Element Antrian " + q_2011);

		int hapus_2011 = q_2011.remove();

		System.out.println("Hapus Elemen = " + hapus_2011);
		System.out.println(q_2011);

		int depan_2011 = q_2011.peek();
		System.out.println("Kepala antrian = " + depan_2011);

		int banyak_2011 = q_2011.size();
		System.out.println("Size antrian = " + banyak_2011);
	}
}
}
```

Pembahasan Class QueueLinkedList:

1. Deklarasi Package

   package pekan4_2511532011;: Menetapkan namespace kelas agar terorganisir dalam direktori praktikum minggu ke-4.

2. Import Queue Interface

   import java.util.Queue;: Mengimpor interface Queue dari library standar Java yang mendefinisikan operasi-operasi dasar antrian.

3. Import LinkedList Class

   import java.util.LinkedList;: Mengimpor kelas LinkedList yang akan digunakan sebagai implementasi konkret dari interface Queue.

4. Deklarasi Kelas

   public class QueueLinkedList_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai program demonstrasi penggunaan Queue dengan LinkedList.

5. Metode Utama (Main)

   public static void main (String [] args): Titik masuk (entry point) program dimana eksekusi kode dimulai.

6. Instansiasi Objek Queue

   Queue<Integer> q_2011 = new LinkedList<>();:

   Membuat objek Queue dengan tipe data Integer menggunakan Generics.

   LinkedList<>() digunakan sebagai implementasi konkret karena LinkedList secara native mengimplementasikan interface Queue.

   Kelebihan LinkedList untuk Queue adalah tidak perlu menentukan kapasitas awal dan operasi enqueue/dequeue lebih efisien (O(1)).

7. Perulangan Penambahan Data (Enqueue)

   for (int i = 0; i < 6; i++): Perulangan berjalan sebanyak 6 kali dengan nilai i dari 0 hingga 5.

   q_2011.add(i);: Metode add() menambahkan elemen ke belakang antrian (enqueue).

   State Antrian: Setelah loop selesai, antrian berisi [0, 1, 2, 3, 4, 5] dengan 0 di depan dan 5 di belakang.

8. Menampilkan Isi Antrian

   System.out.println("Element Antrian " + q_2011);:

   Mencetak seluruh elemen antrian ke konsol.

   Secara otomatis memanggil metode toString() dari LinkedList yang memformat output dengan kurung siku.

   Expected Output: Element Antrian [0, 1, 2, 3, 4, 5].

9. Operasi Remove (Dequeue)

   int hapus_2011 = q_2011.remove();:

   Memanggil metode remove() yang menghapus dan mengembalikan elemen paling depan dari antrian.

Elemen 0 (yang pertama masuk) akan dihapus dan nilainya disimpan ke variabel hapus_2011.

State Antrian: Antrian kini berisi [1, 2, 3, 4, 5].

10. Menampilkan Hasil Dequeue

    System.out.println("Hapus Elemen = " + hapus_2011);: Mencetak nilai elemen yang telah dihapus (nilai 0).

    System.out.println(q_2011);: Mencetak kondisi antrian setelah penghapusan untuk verifikasi.

11. Operasi Peek (Lihat Depan)

    int depan_2011 = q_2011.peek();:

    Memanggil metode peek() yang mengambil nilai elemen paling depan tanpa menghapusnya.

    Setelah operasi remove sebelumnya, elemen depan sekarang adalah 1.

    Nilai 1 disimpan ke variabel depan_2011.

12. Menampilkan Elemen Depan

    System.out.println("Kepala antrian = " + depan_2011);: Mencetak nilai elemen paling depan saat ini.

    Expected Output: Kepala antrian = 1.

13. Operasi Size (Ukuran Antrian)

    int banyak_2011 = q_2011.size();:

    Memanggil metode size() yang mengembalikan jumlah elemen aktual dalam antrian.

    Setelah 1 elemen dihapus dari 6 elemen awal, size sekarang adalah 5.

14. Menampilkan Ukuran Antrian

    System.out.println("Size antrian = " + banyak_2011);: Mencetak jumlah elemen saat ini.

    Expected Output: Size antrian = 5.

15. Perbandingan dengan QueueArray

    Implementasi menggunakan LinkedList lebih sederhana karena tidak perlu mengelola index front, rear, dan capacity secara manual.

    LinkedList secara otomatis menangani alokasi memori dinamis, sehingga tidak ada batasan kapasitas maksimum seperti pada implementasi array.

Metode add(), remove(), peek(), dan size() adalah metode standar dari interface Queue yang konsisten digunakan di berbagai implementasi.

```java
package pekan4_2511532011;
import java.util.Stack;
import java.util.Queue;
import java.util.LinkedList;

public class ReverseData_2511532011 {
	public static void main (String []args) {
		Queue<Integer> q_2011 = new LinkedList<Integer>();

		q_2011.add(1);
		q_2011.add(2);
		q_2011.add(3);

		System.out.println("sebelum reverse " + q_2011 );

		Stack<Integer> s_2011 = new Stack<Integer>();

		while(!q_2011.isEmpty()) {
			s_2011.push(q_2011.remove());
		}

		while(!s_2011.isEmpty()) {
			q_2011.add(s_2011.pop());
		}
		System.out.println("Sesudah reverse= " + q_2011);

	}
}
```

Pembahasan Class ReverseData:

1. Deklarasi Package

   package pekan4_2511532011;: Menetapkan namespace kelas agar terorganisir dalam direktori praktikum minggu ke-4.

2. Import Stack Class

   import java.util.Stack;: Mengimpor kelas Stack dari library standar Java yang mengimplementasikan struktur data tumpukan dengan prinsip LIFO (Last In First Out).

3. Import Queue Interface

   import java.util.Queue;: Mengimpor interface Queue yang mendefinisikan operasi-operasi dasar antrian dengan prinsip FIFO (First In First Out).

4. Import LinkedList Class

   import java.util.LinkedList;: Mengimpor kelas LinkedList yang akan digunakan sebagai implementasi konkret dari interface Queue.

5. Deklarasi Kelas

   public class ReverseData_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai program demonstrasi pembalikan data.

6. Metode Utama (Main)

   public static void main (String []args): Titik masuk (entry point) program dimana eksekusi kode dimulai.

7. Instansiasi Objek Queue

   Queue<Integer> q_2011 = new LinkedList<Integer>();:

   Membuat objek Queue dengan tipe data Integer menggunakan Generics.

   LinkedList digunakan sebagai implementasi konkret untuk menyimpan data antrian.

8. Penambahan Data ke Queue

   q_2011.add(1);: Menambahkan nilai 1 ke belakang antrian.

   q_2011.add(2);: Menambahkan nilai 2 ke belakang antrian.

   q_2011.add(3);: Menambahkan nilai 3 ke belakang antrian.

   State Queue: Antrian berisi [1, 2, 3] dengan 1 di depan dan 3 di belakang.

9. Menampilkan Queue Sebelum Reverse

   System.out.println("sebelum reverse " + q_2011);:

   Mencetak isi antrian sebelum proses pembalikan.

   Expected Output: sebelum reverse [1, 2, 3].

10. Instansiasi Objek Stack

    Stack<Integer> s_2011 = new Stack<Integer>();:

    Membuat objek Stack dengan tipe data Integer untuk membantu proses pembalikan.

    Stack akan memanfaatkan prinsip LIFO untuk membalik urutan FIFO dari Queue.

11. Loop Pertama (Queue ke Stack)

    while(!q_2011.isEmpty()): Perulangan berjalan selama Queue masih memiliki elemen.

    s_2011.push(q_2011.remove());:

    q_2011.remove(): Menghapus dan mengambil elemen paling depan dari Queue (FIFO).

s_2011.push(...): Menambahkan elemen tersebut ke atas Stack (LIFO).

Proses: Elemen 1 masuk Stack dulu, lalu 2, lalu 3.

State Stack: [1, 2, 3] dengan 3 di puncak (top).

State Queue: Kosong [].

12. Loop Kedua (Stack ke Queue)

    while(!s_2011.isEmpty()): Perulangan berjalan selama Stack masih memiliki elemen.

    q_2011.add(s_2011.pop());:

    s_2011.pop(): Menghapus dan mengambil elemen paling atas dari Stack (LIFO).

    q_2011.add(...): Menambahkan elemen tersebut ke belakang Queue (FIFO).

    Proses: Elemen 3 keluar Stack dulu, lalu 2, lalu 1.

    State Stack: Kosong [].

    State Queue: [3, 2, 1] dengan 3 di depan dan 1 di belakang.

13. Menampilkan Queue Setelah Reverse

    System.out.println("Sesudah reverse= " + q_2011);:

    Mencetak isi antrian setelah proses pembalikan selesai.

    Expected Output: Sesudah reverse= [3, 2, 1].

```java
package pekan4_2511532011;
import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;

public class IterasiQueue_2511532011 {
	public static void main (String []args) {
		Queue<String> q_2011 = new LinkedList<>();

		q_2011.add("Pratikum");
		q_2011.add("struktur");
		q_2011.add("Data");
		q_2011.add("Dan");
		q_2011.add("Algoritma");

		Iterator<String> iterator_2011 = q_2011.iterator();

		while(iterator_2011.hasNext()) {
			System.out.print(iterator_2011.next() + " ");
		}
	}
}
```

Pembahasan Class IterasiQueue:

1. Deklarasi Package

   package pekan4_2511532011;: Menetapkan namespace kelas agar terorganisir dalam direktori praktikum minggu ke-4.

2. Import Iterator Interface

   import java.util.Iterator;: Mengimpor interface Iterator dari library standar Java yang menyediakan metode untuk traversing (menelusuri) koleksi elemen satu per satu.

3. Import Queue Interface

   import java.util.Queue;: Mengimpor interface Queue yang mendefinisikan operasi-operasi dasar antrian.

4. Import LinkedList Class

   import java.util.LinkedList;: Mengimpor kelas LinkedList yang akan digunakan sebagai implementasi konkret dari interface Queue.

5. Deklarasi Kelas

   public class IterasiQueue_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai program demonstrasi iterasi Queue.

6. Metode Utama (Main)

   public static void main (String []args): Titik masuk (entry point) program dimana eksekusi kode dimulai.

7. Instansiasi Objek Queue

   Queue<String> q_2011 = new LinkedList<>();:

   Membuat objek Queue dengan tipe data String menggunakan Generics.

   LinkedList<> digunakan sebagai implementasi konkret.

   Variabel q_2011 adalah referensi untuk mengakses metode-metode antrian.

8. Penambahan Data ke Queue (Enqueue)

   q_2011.add("Pratikum");: Menambahkan string "Pratikum" ke belakang antrian.

   q_2011.add("struktur");: Menambahkan string "struktur" setelah "Pratikum".

   q_2011.add("Data");: Menambahkan string "Data" setelah "struktur".

   q_2011.add("Dan");: Menambahkan string "Dan" setelah "Data".

   q_2011.add("Algoritma");: Menambahkan string "Algoritma" setelah "Dan".

   State Queue: Antrian berisi [Pratikum, struktur, Data, Dan, Algoritma] dengan urutan FIFO (First In First Out).

9. Inisialisasi Iterator

   Iterator<String> iterator_2011 = q_2011.iterator();:

   Memanggil metode iterator() pada objek Queue.

   Mengembalikan objek Iterator yang diposisikan di awal elemen Queue (sebelum elemen pertama).

   Iterator ini memungkinkan traversal elemen tanpa perlu mengakses index secara langsung.

10. Looping Penelusuran (Traversal)

    while(iterator_2011.hasNext()):

    Struktur perulangan yang berjalan selama iterator masih memiliki elemen berikutnya.

    Metode hasNext() mengembalikan true jika masih ada elemen yang belum diakses, dan false jika sudah mencapai akhir.

11. Eksekusi Next dan Print

    System.out.print(iterator_2011.next() + " ");:

    iterator_2011.next(): Mengambil elemen saat ini dan menggeser posisi iterator ke elemen berikutnya.

    System.out.print(...): Mencetak elemen tersebut ke konsol diikuti dengan spasi tanpa pindah baris (newline).

    Proses ini berulang hingga semua elemen dalam Queue habis diakses.

2.2 Pembahasan Tugas Pratikum

```java
package pekan4_2511532011;
import java.util.Scanner;

public class AntrianLoket_2511532011 {

    int front_2011;
    int rear_2011;
    int max_2011;
    String queue_2011[];

    public AntrianLoket_2511532011(int kapasitas) {
        this.max_2011 = kapasitas;
        this.queue_2011 = new String[max_2011];
        this.front_2011 = 0;
        this.rear_2011 = -1;
    }


    public boolean isEmpty() {
        return rear_2011 < front_2011;
    }

    public boolean isFull() {
        return rear_2011 == max_2011 - 1;
    }

    public void enqueue(String data) {
        if (isFull()) {
            System.out.println("Antrian penuh!");
            return;
        }
        rear_2011++;
        queue_2011[rear_2011] = data;
        System.out.println("Data berhasil ditambahkan ke antrian");
    }


    public void dequeue() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println(queue_2011[front_2011] + " telah dilayani");
        front_2011++;

        if (isEmpty()) {
            front_2011 = 0;
            rear_2011 = -1;
        }
    }

    public void display() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("Isi antrian:");
        for (int i = front_2011; i <= rear_2011; i++) {
            System.out.println((i - front_2011 + 1) + ". " + queue_2011[i]);
        }
    }
```

```java
    public void reverse() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        int left = front_2011;
        int right = rear_2011;

        while (left < right) {
            String temp = queue_2011[left];
            queue_2011[left] = queue_2011[right];
            queue_2011[right] = temp;
            left++;
            right--;
        }

    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        AntrianLoket_2511532011 antrian = new AntrianLoket_2511532011(10);

        int pilihan_2011 = 0;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");

            if (scanner.hasNextInt()) {
                pilihan_2011 = scanner.nextInt();
                scanner.nextLine();
            } else {
                System.out.println("Input tidak valid. Masukkan angka.");
                scanner.nextLine();
                continue;
            }

            switch (pilihan_2011) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = scanner.nextLine();
                    antrian.enqueue(nama);
                    break;
                case 2:
                    antrian.dequeue();
                    break;
                case 3:
                    antrian.display();
                    break;
                case 4:
                    antrian.reverse();
                    break;
                case 5:
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
                    break;
```

```
            }
        } while (pilihan_2011 != 5);

        scanner.close();
    }
}
```

Pembahasan AntrianLoket:

1. Deklarasi Kelas dan Atribut

   public class AntrianLoket_2511532011: Kelas utama yang berfungsi sebagai cetak biru untuk objek antrian loket.

   int front_2011;: Index yang menunjuk ke elemen terdepan (antrian pertama yang akan dilayani).

   int rear_2011;: Index yang menunjuk ke elemen terbelakang (antrian terakhir yang masuk).

   int max_2011;: Variabel penyimpan kapasitas maksimum antrian.

   String queue_2011[]: Array bertipe String yang berfungsi sebagai wadah penyimpanan fisik data antrian.

2. Konstruktor (Inisialisasi)

   public AntrianLoket_2511532011(int kapasitas): Konstruktor yang menerima parameter ukuran antrian.

   this.max_2011 = kapasitas;: Assign nilai kapasitas ke variabel instance.

   this.queue_2011 = new String[max_2011];: Alokasi memori array dengan panjang sesuai kapasitas.

   this.front_2011 = 0;: Inisialisasi index depan di posisi 0.

   this.rear_2011 = -1;: Inisialisasi index belakang di -1, menandakan state awal antrian benar-benar kosong.

3. Metode isEmpty()

   return rear_2011 < front_2011;: Logika pemeriksaan antrian kosong. Ketika elemen dihapus terus-menerus, front akan bergeser melewati rear. Jika rear lebih kecil dari front, berarti tidak ada data valid di antrian.

4. Metode isFull()

   return rear_2011 == max_2011 - 1;: Memeriksa apakah antrian sudah penuh dengan mengecek apakah index belakang sudah menyentuh batas maksimum array. (Catatan: Ini adalah antrian linear, bukan circular).

5. Metode enqueue (Masuk Antrian)

   if (isFull()) { ... return; }: Guard clause. Jika array penuh, cetak pesan dan hentikan eksekusi metode.

   rear_2011++;: Increment index belakang untuk mempersiapkan slot kosong berikutnya.

   queue_2011[rear_2011] = data;: Simpan string nama pelanggan ke dalam array pada posisi rear yang baru.

   Cetak konfirmasi keberhasilan penambahan.

6. Metode dequeue (Keluar/Layani Antrian)

   if (isEmpty()) { ... return; }: Guard clause. Jika kosong, cetak pesan dan hentikan.

   System.out.println(queue_2011[front_2011] + " telah dilayani");: Tampilkan elemen terdepan sebagai tanda telah dilayani.

   front_2011++;: Geser index depan ke kanan. Secara logis, elemen sebelumnya dianggap terhapus karena tidak lagi berada dalam rentang front hingga rear.

   if (isEmpty()) { front_2011 = 0; rear_2011 = -1; }: Reset Logis. Penting untuk mencegah index front terus bergeser ke kanan hingga melampaui kapasitas array. Saat antrian benar-benar kosong, index dikembalikan ke kondisi awal.

7. Metode display (Tampilkan Antrian)

   if (isEmpty()) { ... return; }: Cek kondisi kosong sebelum mencetak.

   for (int i = front_2011; i <= rear_2011; i++): Loop iterasi hanya pada elemen yang valid (antara front dan rear).

   System.out.println((i - front_2011 + 1) + ". " + queue_2011[i]);: Mencetak elemen dengan penomoran urut relatif (1, 2, 3...) agar tampilan lebih rapi dan sesuai konteks antrian manusia.

8. Metode reverse (Balik Urutan)

   if (isEmpty()) { ... return; }: Cek kondisi kosong.

   int left = front_2011; int right = rear_2011;: Inisialisasi dua pointer di ujung kiri dan kanan data valid.

while (left < right): Perulangan berjalan selama kedua pointer belum bertemu di tengah.

String temp = ...; queue[left] = queue[right]; queue[right] = temp;: Teknik swap untuk menukar nilai elemen.

left++; right--;: Geser pointer menuju tengah. Algoritma ini membalik urutan secara in-place dengan kompleksitas ruang O(1).

9. Metode main & Inisialisasi Objek

   Scanner scanner = new Scanner(System.in);: Aktifkan scanner untuk input konsol.

   AntrianLoket_2511532011 antrian = new AntrianLoket_2511532011(10);: Buat instance antrian dengan kapasitas 10 loket/pelanggan.

   int pilihan_2011 = 0;: Variabel penampung pilihan menu dari pengguna.

10. Loop Menu & Validasi Input

    do { ... } while (pilihan_2011 != 5);: Struktur loop yang memastikan menu tampil minimal sekali dan berulang hingga user memilih keluar.

    if (scanner.hasNextInt()): Validasi tipe input. Mencegah program crash (InputMismatchException) jika user memasukkan huruf/simbol.

    scanner.nextLine(); (di blok else): Membersihkan buffer input yang salah agar loop tidak menjadi infinite.

    pilihan_2011 = scanner.nextInt(); scanner.nextLine();: Baca angka pilihan dan konsumsi karakter \n tersisa agar input nextLine() berikutnya tidak terlewat.

11. Logika switch-case

    case 1: Prompt nama pelanggan, baca input string, panggil enqueue().

    case 2: Panggil dequeue() untuk melayani antrian terdepan.

    case 3: Panggil display() untuk mencetak seluruh antrian saat ini.

    case 4: Panggil reverse() untuk membalik urutan antrian secara langsung.

    case 5: Cetak pesan terminasi program.

    default: Tangani angka di luar 1-5 dengan pesan error.

BAB III
KESIMPULAN

3.1 Kesimpulan

ADT Queue merupakan struktur data linear yang menerapkan prinsip FIFO (First In First Out), di mana elemen yang pertama kali masuk akan selalu menjadi elemen pertama yang dikeluarkan. Dalam bahasa pemrograman Java, Queue diimplementasikan melalui antarmuka java.util.Queue dengan kelas pendukung seperti LinkedList atau ArrayDeque yang menyediakan metode utama seperti add(), poll(), dan peek() untuk manipulasi data yang terstruktur. Mekanisme kerja Queue membatasi akses data hanya pada dua ujung berbeda, sehingga mencegah konflik indeks dan menjamin urutan pemrosesan yang konsisten serta aman. Karakteristik ini menjadikan Queue sangat efisien untuk menyelesaikan berbagai permasalahan komputasi nyata, mulai dari penjadwalan sistem, algoritma Breadth-First Search, hingga manajemen buffer dan simulasi antrian layanan. Melalui praktikum ini, pemahaman konseptual mengenai struktur data FIFO serta kemampuan teknis dalam mengimplementasikan metode Queue pada Java telah tercapai secara optimal sebagai fondasi pengembangan aplikasi yang lebih kompleks..

DAFTAR PUSTAKA

[1]W3schools,“JavaDataStructur”2026[Daring].Tersediapada:https://www.w3schools.com/java/java_data_structures.asp.[Diakses:25-april-2026]